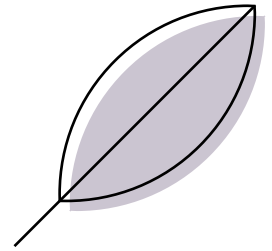
PREDICTING TAX BURDEN FOR SMARTER INVESTMENTS

INTENDED AUDIENCE : REAL ESTATE FIRMS ,BUYERS & SELLERS

By – Poorvi Raut



INTRODUCTION

- In real estate, a property's listed price doesn't reveal its full cost—**property taxes** can quietly reduce profits, especially for rental or flip investments.
- Our ML model predicts the **tax-to-price ratio** using features like location, size, and amenities—helping investors spot tax-heavy properties upfront.
- Using **geographic encoding** and **multivariate regression**, we highlight low-tax, high-value opportunities for smarter, more profitable decisions.

DATASET OVERVIEW

Variable	Description
MLS	Unique Identifier for each property listing
sold_price	Final selling price of the property
zipcode	ZIP code where the property is located
longitude	Longitude coordinate of the property
latitude	Latitude coordinate of the property
lot_acres	Size of the lot in acres
taxes	Property taxes in USD
year_built	Year the property was constructed
bedrooms	Number of bedrooms in the property
bathrooms	Number of bathrooms in the property
sqrt_ft	Total square footage of the property
garage	Number of garage spaces
kitchen_features	Features available in the kitchen (e.g., Dishwasher, Refrigerator)
fireplaces	Number of fireplaces
floor_covering	Types of floor coverings used in the property
HOA	Homeowners Association fees (if applicable)
HOA_category	Categorization of HOA fees (e.g., Low, Medium, High, None)
price_per_sqft	Calculated price per square foot (sold_price / sqrt_ft)
tax_price_ratio	Ratio of property taxes to the selling price (taxes / sold_price)

- **Source:** Modified from **raw_house_data**
  - **Original:** 5,000 rows × 16 columns (price, location, size, beds, baths, etc.)

- **Cleaned Dataset** after data cleaning and outliers removal **: cleaned_house_data**
  - **5000 rows × 19 columns** (includes engineered features)

Key Engineered features :

- **tax_price_ratio** = annual taxes / sale price
- **price_per_sqft** = sale price / square footage
- **HOA_category** = Categorization of HOA fees (e.g., Low, Medium, High, None)

DATA PRE-PROCESSING

- Cleaned dataset (missing values handled), EDA performed, and new features engineered: price_per_sqft and tax_price_ratio were binned into 20 categories each, creating **price_category and tax_category.**
- **pd.qcut()** in pandas was used to create equal-sized bins for categorization. Each bin has roughly the same number of houses

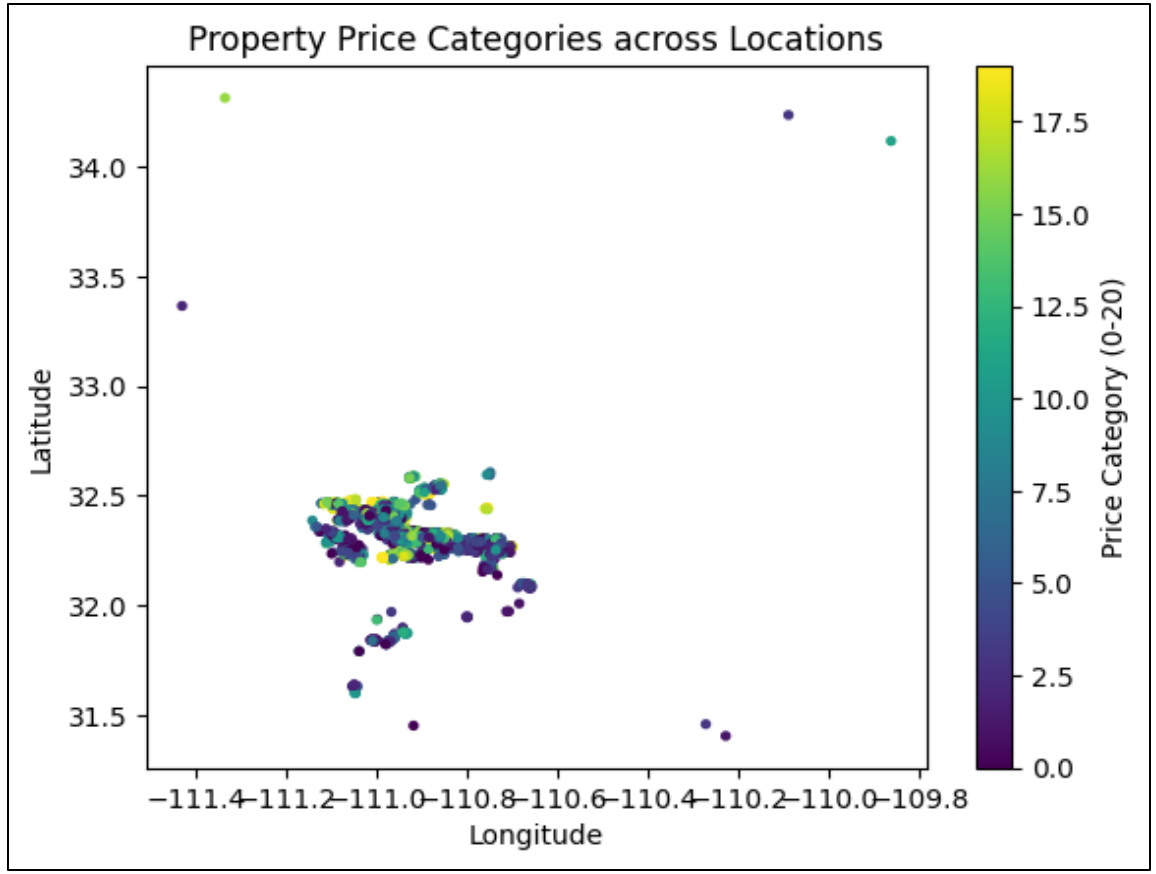
	price_per_sqft	price_category
0	533.372420	19
1	475.007308	19
2	199.584200	9
3	280.237642	18
4	501.543210	19

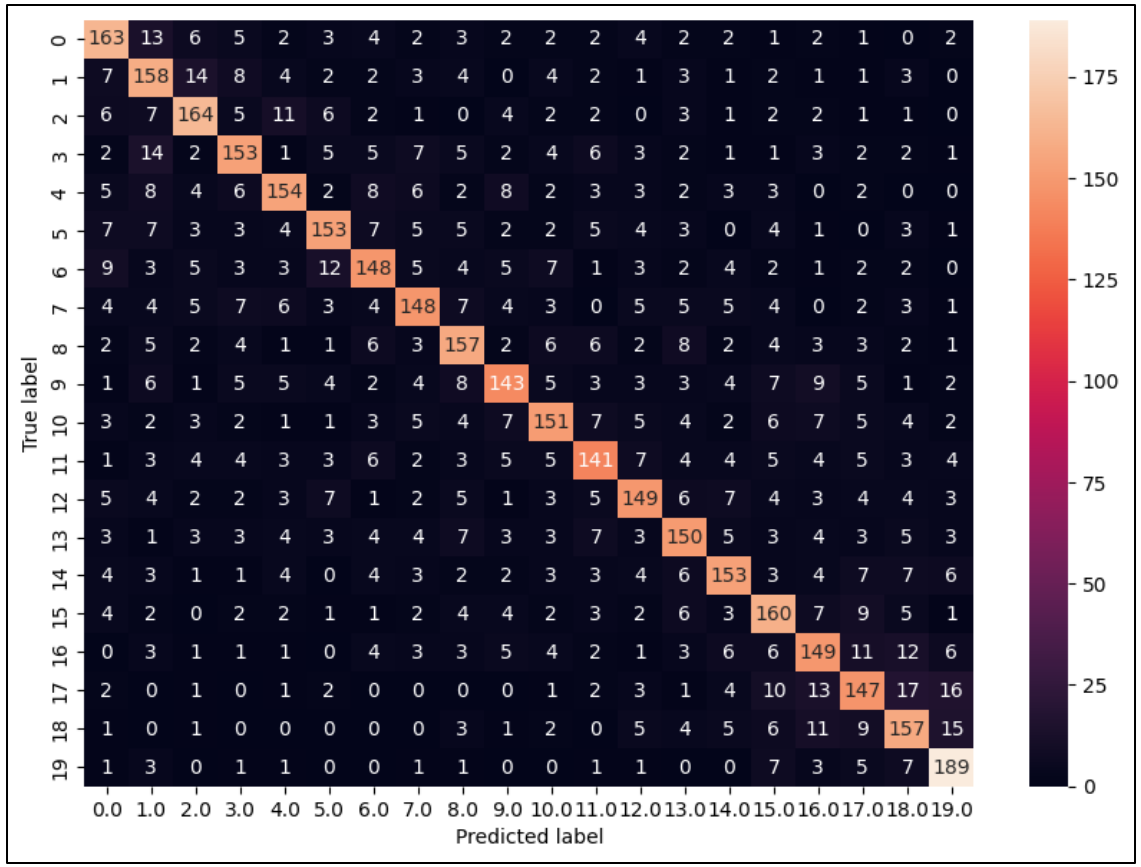
	tax_price_ratio	tax_category
0	0.004512	1
1	0.008555	8
2	0.007933	6
3	0.008658	8
4	0.005826	2

Model 1: GEOGRAPHIC ENCODER USING KNN

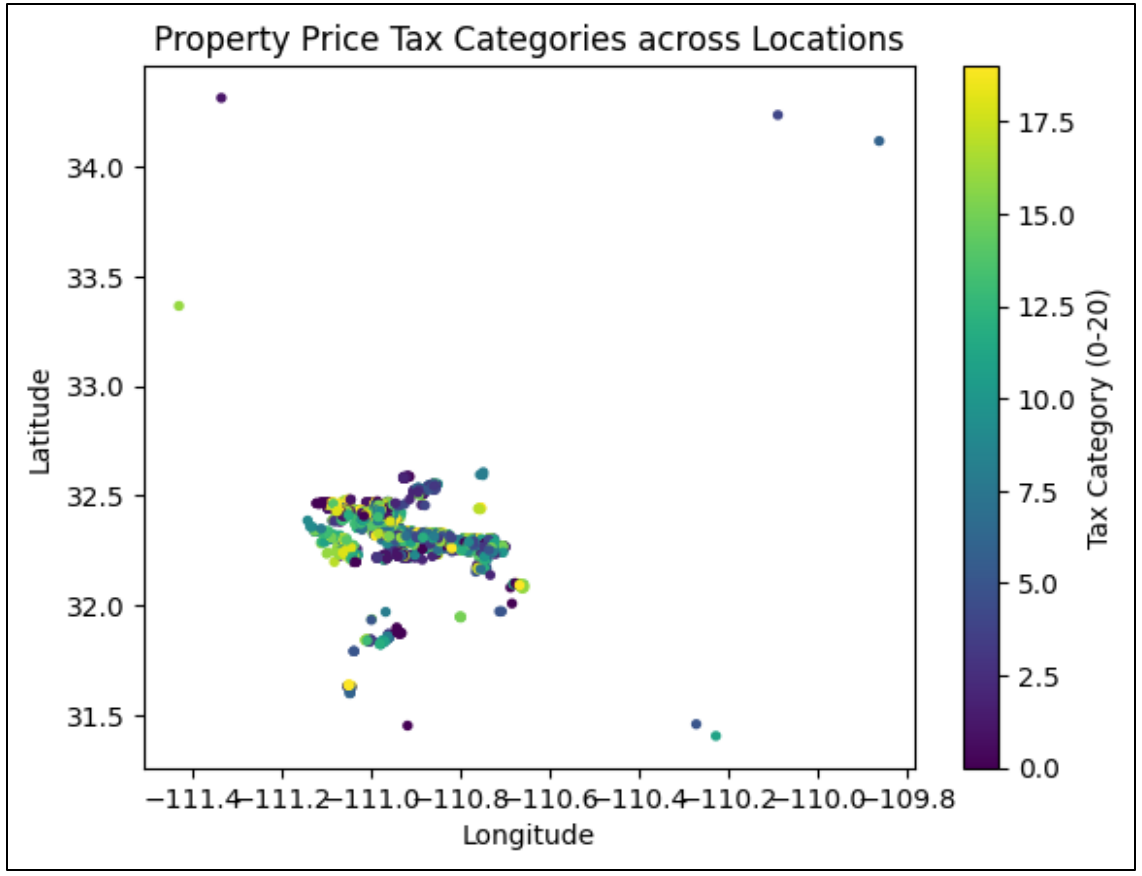
- A geographic encoder using a KNN classifier is a method to predict or categorize the geographic location (e.g., region, city) of a data point based on its other features.
- Using longitude and latitude (X) and KNN, we predict price_category and tax_category (targets).
- This tells us, based on location, if an area tends to have high/low priced houses or high/low property taxes, providing a simplified geographic understanding of these economic indicators.

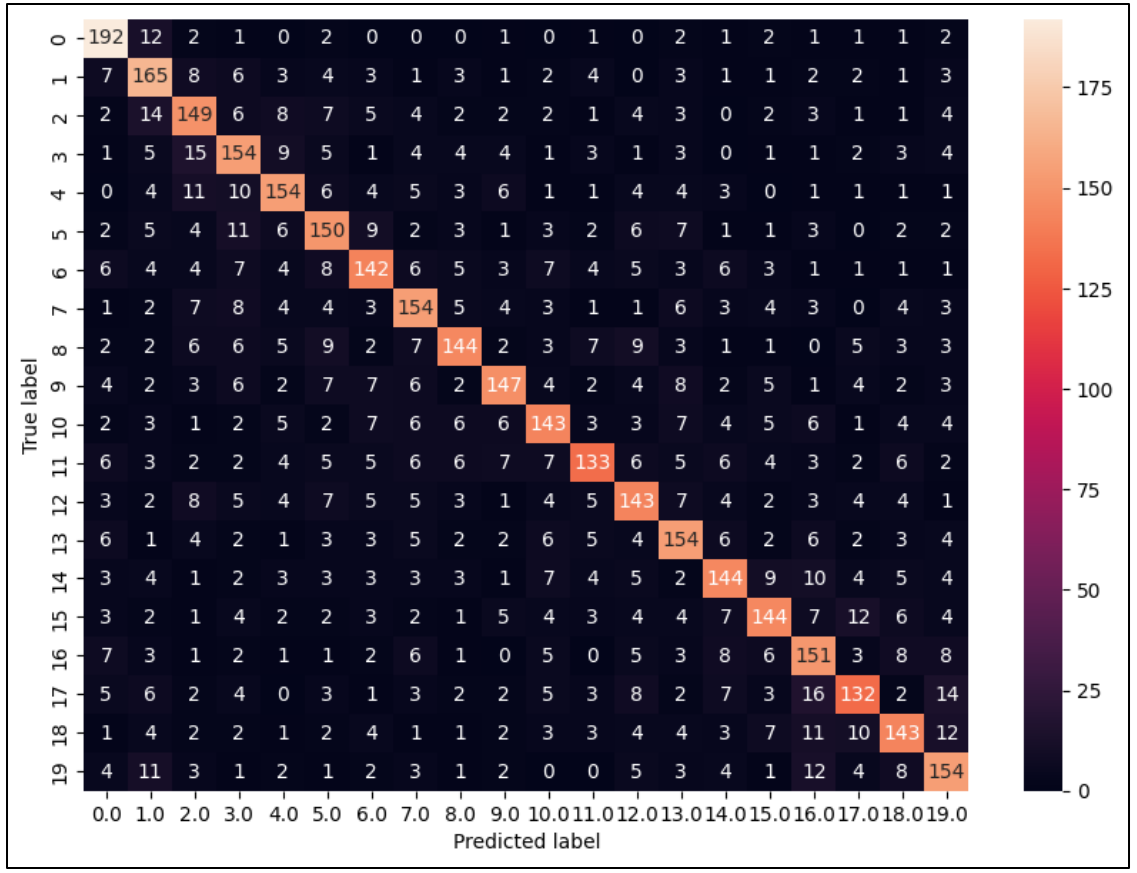
X: Longitude and Latitude vs Y: price_category





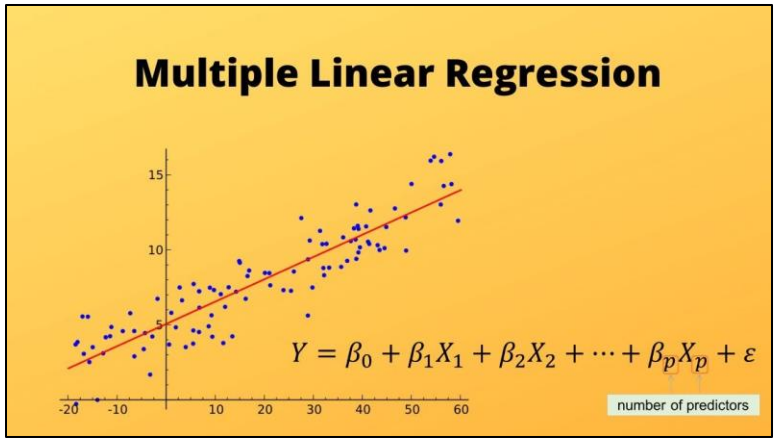
X: Longitude and Latitude vs Y: tax_category

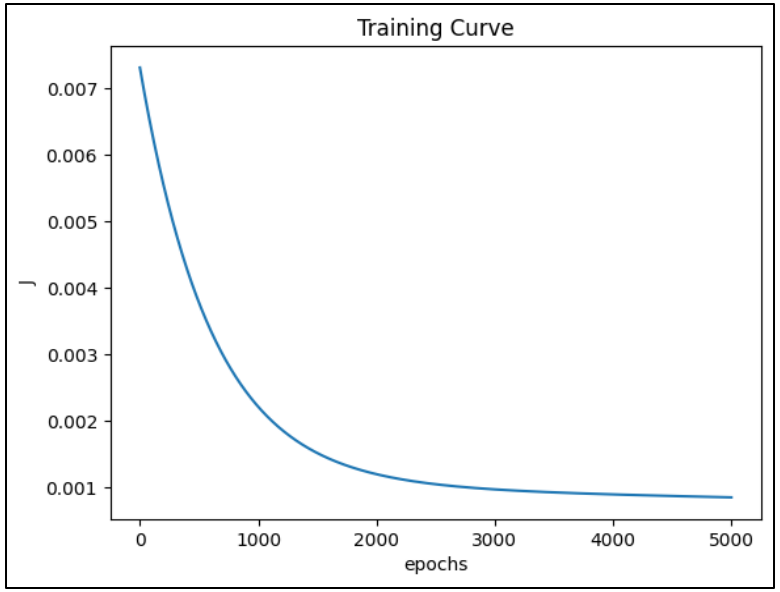




Model 2: Multivariate Linear Regressor

- Multivariate Linear Regression models the linear relationship between a dependent variable and **multiple** independent variables to make predictions.
- Predicts an outcome (Y) using multiple factors (X) with the formula: $\hat{\mathbf{Y}} = \boldsymbol{\beta_0} + \boldsymbol{\beta_1 X_1} + \boldsymbol{\beta_2 X_2} + \ldots + \boldsymbol{\beta_p X_p}$
- It learns the best "weights" (βs) for each factor by minimizing prediction errors (MSE) difference between predicted (Ŷ) and actual (Y) values over many tries ("epochs"), adjusting these weights based on a "learning rate."
- For our housing dataset we are trying to find a linear relationship between our dependent variable Y : **tax_price_ratio** and X: independent vaiables as **sold_price**, **sqrt_ft** and **taxes**





Why target as tax_price_ratio ??

- It tells you **how much annual property tax** you pay **relative** to the **property's sale price.**

$$\text{tax\_price\_ratio} = \frac{\text{taxes}}{\text{sold\_price}}$$

- As a data scientist working for a real estate investment firm I wanted to identify overvalued properties w.r.t undervalued ones— homes that have low tax burdens relative to their value.

- By predicting tax_price_ratio, the firm can filter out high-tax homes and recommend the most tax-efficient investments to clients.

•This ratio tells you how "tax-heavy" a property is.
•Lower ratio = better investment
•Higher ratio = hidden costs (eg. Auto insurance company)

Why Independent Variables as : **sold_price, taxes and sqrt_ft**

```
[ ]  X_raw = df[['tax_price_ratio','sold_price','sqrt_ft','taxes']].to_numpy()
```

- **Higher sold_price:** Tends to **decrease** the tax_price_ratio (the denominator increases more than the numerator).
- **Higher taxes:** Directly **increases** the tax_price_ratio (the numerator increases).
- **Higher sqrt_ft:** Indirect effect, likely **decreases** the tax_price_ratio if larger homes correlate with higher selling prices that outpace tax increases.

- Adding Bedrooms, Bathrooms, and Lot_Acres with their different value ranges and different scales could negatively impact the model's training and performance..

Feature Scaling of Independent features

- **Feature scaling** makes all features equally important for the model, leading to better learning and faster results.
- For our independent features we employed **Min-max scaling** for all of X values to be in equal range for better prediction of our model

```
[ ]  X_raw = df[['tax_price_ratio','sold_price','sqrt_ft','taxes']].to_numpy()
```

```
[ ]  #  Standardize X
     # 1. Min-Max Scaling
     X_min = X_raw.min(axis=0)
     X_max = X_raw.max(axis=0)
     X_scaled = (X_raw - X_min) / (X_max - X_min)
```

```
[ ]  # 2. Split into target (y) and features (X)
     y = X_scaled[:, 0]        # tax_price_ratio
     X = X_scaled[:, 1:]       # other features
```

Model Evaluation

- After applying feature scaling and normalization, we train our model, test the predicted values, and evaluate performance metrics to gain deeper insights into its accuracy and effectiveness.

- I adjusted the learning rate and training time (epochs) to improve predictions for calculating the mean predictions for three houses.

- Finally calculated the values oof evaluation metrics:

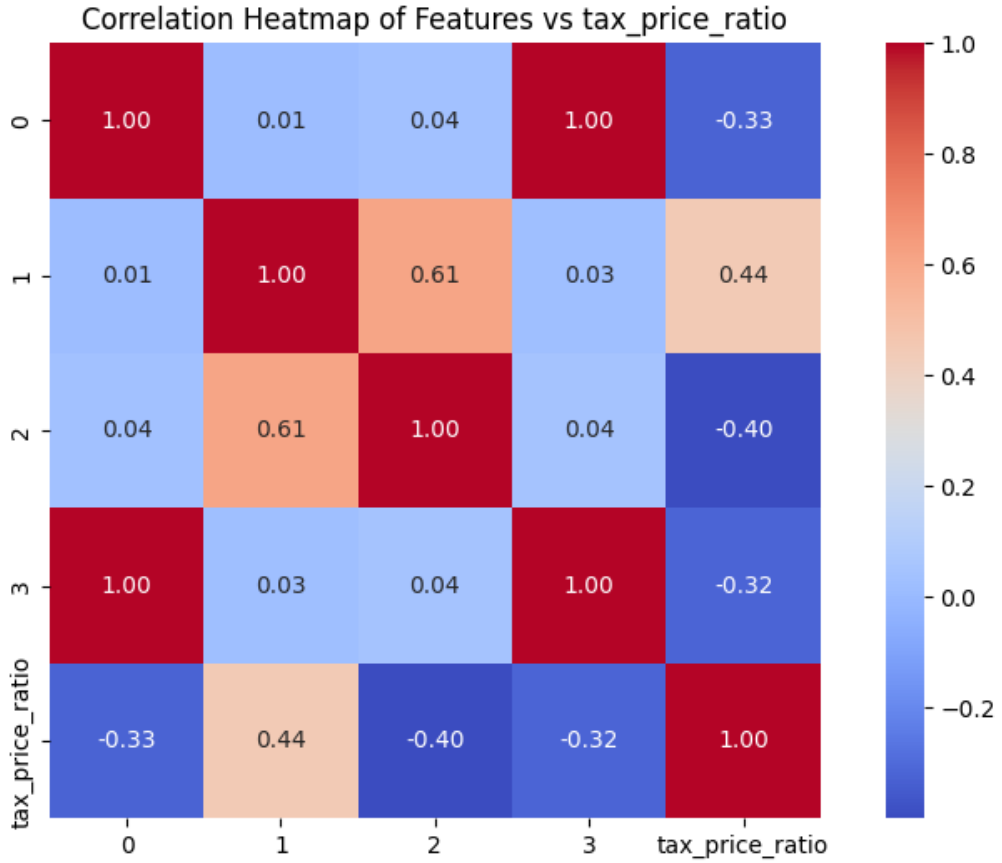
•**MAE:** Average size of prediction errors.
•**R^2 Score:** How well the model fits the data (1 is perfect, negative is poor).
•**OLS Loss:** Average squared difference between predictions and actual values (aim is zero).

```
--- Predictions ---
House 1 Predicted Tax Price Ratio: 0.11398
House 2 Predicted Tax Price Ratio: 0.13916
House 3 Predicted Tax Price Ratio: 0.08343

--- Evaluation on Test Data ---
True Tax Price Ratios: [0.09439 0.008   0.0027 ]
Predicted Tax Price Ratios: [0.11 0.14 0.08]
Mean Prediction: 0.11
MAE: 0.08
R² Score: -0.24
OLS Loss: 0.0
```

Correlation Heatmap

Feature **2** : **taxes** might be the best at identifying **lower tax burden** homes, while **Feature 1:sold_price** could be influencing **higher tax ratios**.



CONCLUSION AND FUTURE ENHANCEMENTS

- We built a **geographic encoder KNN** and **multivariate linear regressor** models to predict a property's tax-to-price ratio, helping identify tax-heavy investments early based on location and other factors.
- While predictions are promising, the current R^2 score (-0.24) shows there's room to improve accuracy.
- Future improvements: More features , better model like polynomial regression, tree-based methods like Random Forest, focus on key data.

Appendix

The visuals are sourced from Google Search, excel and the code was created using the Google Colab IDE.

Dataset Source : **raw_house_data.csv**

THANK YOU